



Ariadne

3D Dungeon Maker

イベント作成ガイド

© Explorers' Lab

Ver 1.5.1

概要	3
Ariadneにおけるイベント	4
<i>EventMasterData</i>	4
<i>EventParts</i>	4
<i>EventProcessData</i>	4
イベントのカスタマイズ	5
スクリプトの作成	5
スクリプトのアタッチ.....	7
<i>EventCategoryEditor</i> での設定.....	9

Chapter 1

概要

このドキュメントはAriadne 3D Dungeon Makerでダンジョン内のイベントを作成したり、カスタマイズされたイベントスクリプトを作成する方法について説明するものです。

イベント用スクリプトを作成し、そのスクリプトをアタッチしたPrefabオブジェクトを作成しておくことで、EventCategoryEditorで対象のイベントを定義することができます。

また、各イベントカテゴリーでは引数を設定することができます。この引数はEventEditorで値を設定可能で、イベント実行時にイベントスクリプトに渡されるので、スクリプトの中で必要な引数については事前にEventCategoryEditorで定義してください。

Ariadneにおけるイベント

Ariadneでは3Dダンジョン内で扉を開けたり宝箱を開けたりするイベントを作成することができます。

イベントのデータ構造は以下のようになっています。

EventMasterData

EventMasterDataはイベントに関するデータを保持するファイルです。このデータはダンジョン内の1つのマスにつき1つセットすることができます。

EventMasterDataはEventPartsと呼ばれるイベントのパーツデータを複数持つことができます。これらのEventPartsではどのEventPartsを実行するかを条件を設定することができます。

EventParts

EventPartsはどのイベントプロセスを実行するかを条件を保持しているデータです。該当のマスに関連づけられたEventMasterDataがある場合、その中に含まれるEventPartsを確認して条件を満たしているパーツがあればイベントプロセスを実行します。このプロセスはEventProcessDataという単位で定義されており、リスト形式で保持することができます。

EventProcessData

EventProcessDataはイベントの処理を行う単位で、スクリプトを使用してイベントの動作を定義します。各EventProcessDataではひとつのイベントカテゴリーが設定されています。イベントカテゴリーでは対応するイベントスクリプトの設定や、引数の型の定義を行うことができ、EventEditor内で引数の値を設定することができます。

イベントのカスタマイズ

この章ではAriadneでイベントを追加する方法について説明します。

スクリプトの作成

イベントの動作を作成するために、イベント用のスクリプトを作成します。イベントスクリプトを作成する際はAriadneEventStrategyBaseを継承し、IAriadneEventのインタフェースを実装してください。

スクリプトを作成する際はデモ用のイベントスクリプトを複製してカスタマイズすると便利です。

例として次のページに "EventWait" のスクリプトを記載します。

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

namespace Ariadne
{
    /// <Summary>
    /// Event process of wait.
    /// </Summary>
    public class EventWait : AriadneEventStrategyBase, IAriadneEvent
    {
        [SerializeField]
        string argNameWaitTime = "WaitTime";

        /// <Summary>
        /// Execute method of event process.
        /// </Summary>
        /// <param name="callbackObj">GameObject which receives call.</param>
        /// <param name="eventPos">The position of the event.</param>
        /// <param name="processData">The event process data.</param>
        /// <param name="debugMode">The flag for debug.</param>
        public void ExecuteAriadneEvent(GameObject callbackObj, Vector2Int eventPos, EventProcessData
processData, bool debugMode)
        {
            PreEventProcess(callbackObj, debugMode);

            string argValue = processData.eventArgs.Find(arg => arg.argName ==
argNameWaitTime).argListValue[0];
            float waitTime = 0f;
            bool success = float.TryParse(argValue, out waitTime);
            if (!success)
            {
                Debug.LogWarning("Event Position : " + eventPos + " / Argument : " + argNameWaitTime
+ " has invalid data. Check your event data in EventEditor.");
            }

            StartCoroutine(WaitProcess(waitTime));
        }

        IEnumerator WaitProcess(float waitTime)
        {
            yield return new WaitForSeconds(waitTime);
            PostEventProcess();
        }
    }
}

```

ゲームの実行中、EventProcessorがこのスクリプトをアタッチしているオブジェクトをインスタンス化し、ExecuteAriadneEvent()のメソッドを実行します。

ExecuteAriadneEvent()のメソッドでは最初にPreEventProcess()のメソッドを実行してください。このメソッドではイベント終了後のコールバックオブジェクトの設定(通常はAriadneEventProcessorのオブジェクト)や、デバッグモードの設定を行います。

次にEventProcessDataに含まれている引数のデータを取得します。サンプルではEventCategoryEditorで指定した引数の名前を使ってFind()メソッドで検索を行なっています。検索のキーとなる文字列はフィールドとして定義し、[SerializeField]の属性をつけることでインスペクターウィンドウから変更できるようにしています。

引数のフィールドはListとして定義されており、EventCategoryEditorで”Is List”にチェックが入っている場合は複数の値が、チェックが入っていない場合は要素数1のリストとなっています。

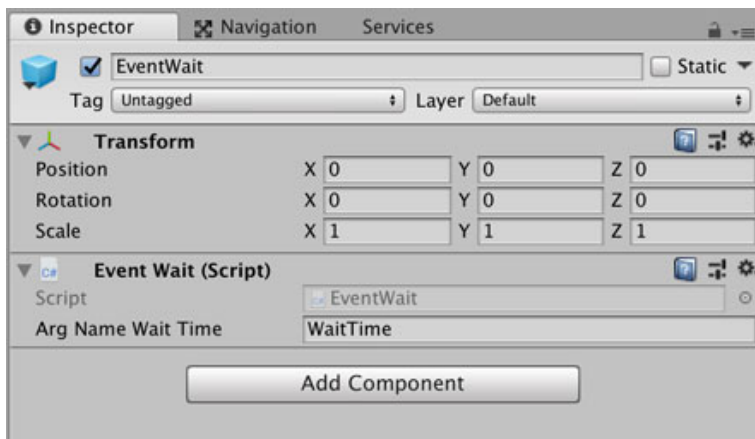
値型の引数については文字列に変換して保持しているため、各型のTryParse()メソッドなどを利用して変換するようにしてください。サンプルではウェイト処理の待ち時間をfloat型の値として取得しています。参照型の場合は対象のオブジェクトへの参照を保持しています。

引数の取得が完了したら任意の処理を行ってください。

処理が完了したら、PostEventProcess()のメソッドを実行することでイベントプロセスの完了がコールバックオブジェクト(EventProcessor)に通知されます。

スクリプトのアタッチ

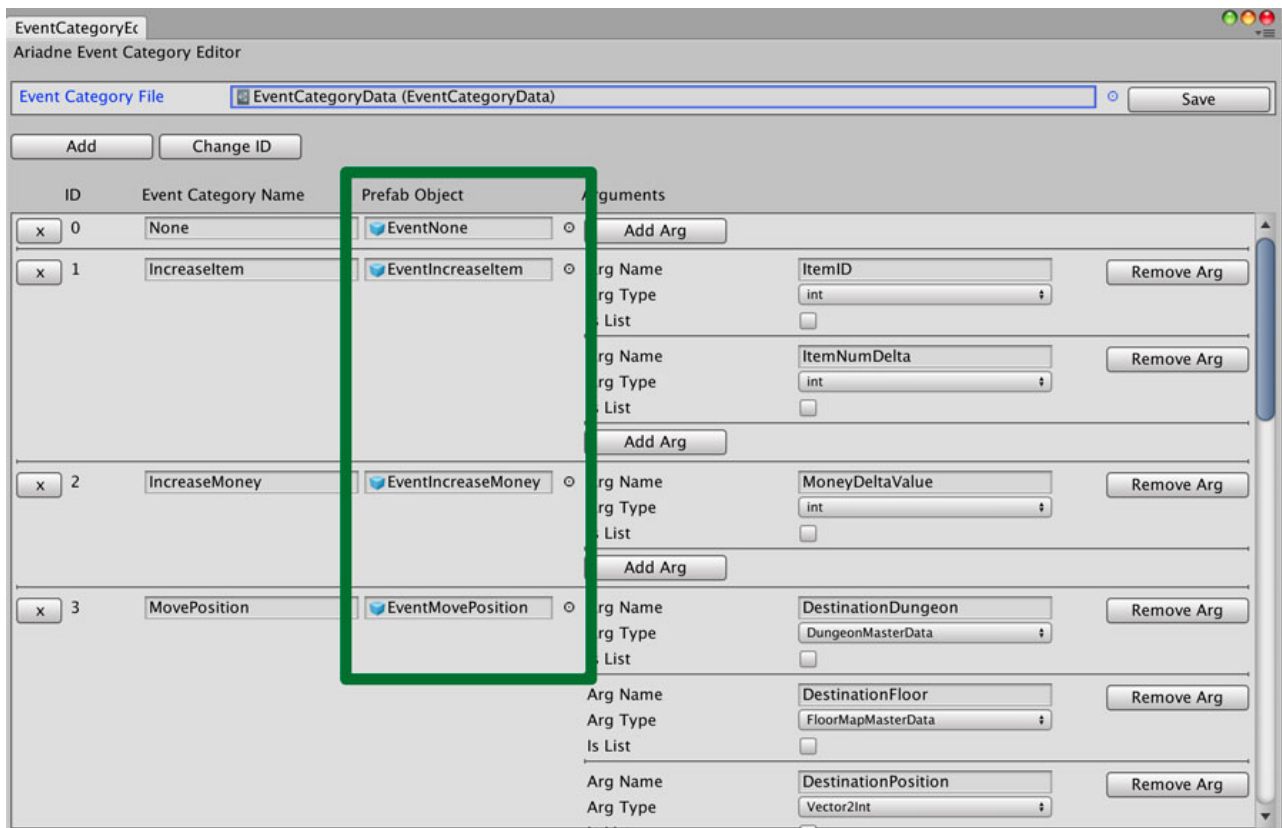
スクリプトの作成後は、空のゲームオブジェクトにスクリプトをアタッチしてPrefab化します。例えば上でサンプルコードとして紹介した”EventWait”のスクリプトについては以下のようにオブジェクトにアタッチされています。



作成したPrefabは任意のフォルダに配置できます。デモ用のイベントPrefabについては[Ariadne/Prefabs/EventObjects] に配置されています。

EventCategoryEditorでの設定

作成したPrefabはEventCategoryEditorでイベント用のオブジェクトとしてセットします。



EventCategoryEditorではイベント用のスクリプトに渡す引数を設定します。ここで指定する名前はイベント用スクリプトで使用する名前と同じものを設定してください。

EventCategoryEditorの詳細についてはマニュアルをご参照ください。